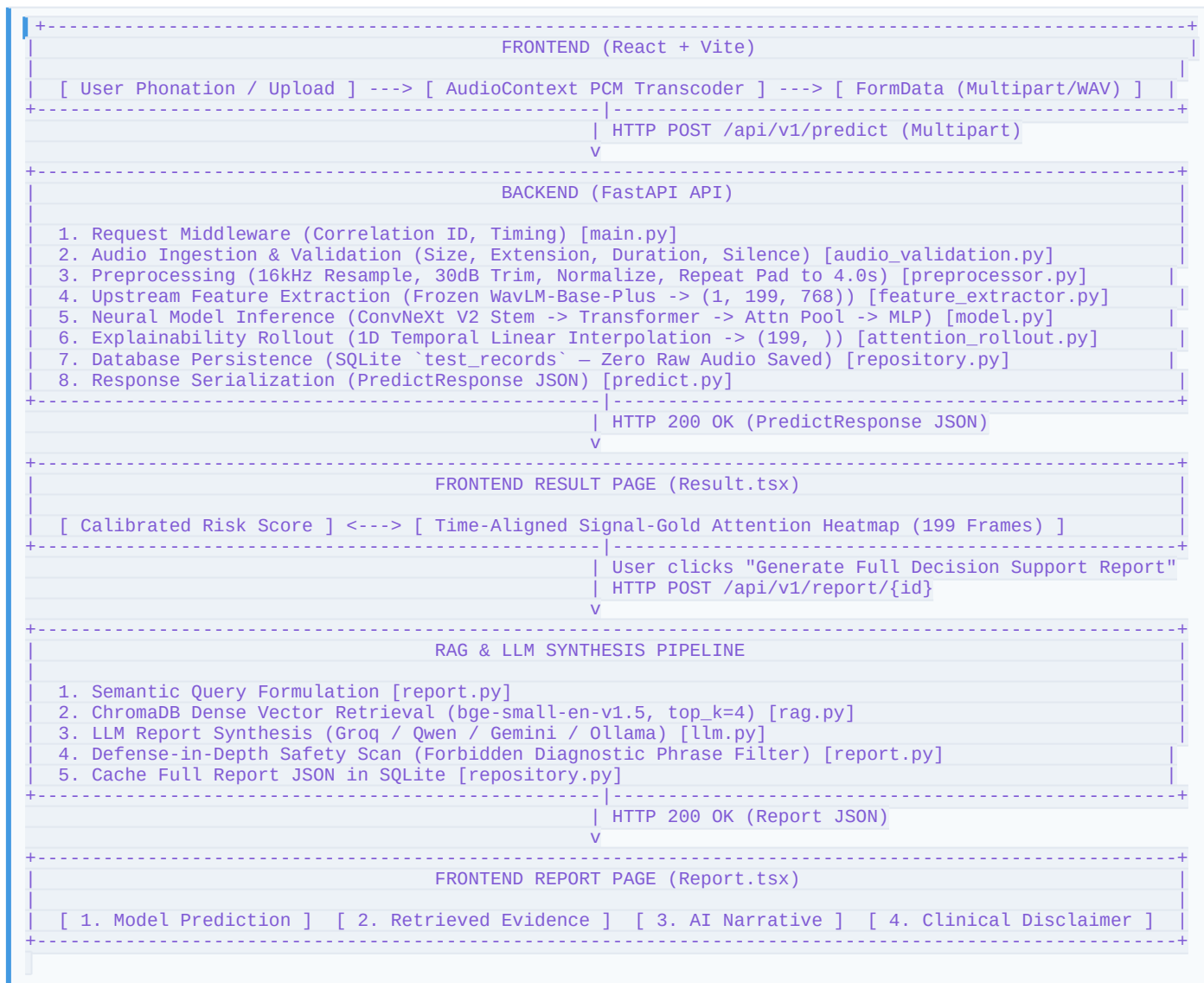


Comprehensive Technical Project Walkthrough & System Architecture Guide

Classification: Research & Screening Decision Support Tool — **Not a Medical Diagnostic Device.**



- **Executing Files:** `frontend/src/components/AudioRecorder.tsx`, `frontend/src/components/UploadDropzone.tsx`, `frontend/src/api/client.ts`
- **Underlying Technology:** Web Audio API (`AudioContext`, `AnalyserNode`), `MediaRecorder` API, HTML5 Canvas API, standard Fetch API (`FormData`).

- **Input:** User microphone input stream (live sustained vowel phonation /a/) or uploaded audio file (`.wav` , `.mp3` , `.m4a` , `.ogg` , `.webm`).
- **Processing Logic:**
 1. For live recording, `AudioRecorder.tsx` connects the audio stream to an `AnalyserNode` (`fftSize = 2048`) to render a real-time oscilloscope waveform on an HTML5 canvas in `#D9A55C` (signal-gold).
 2. Upon stopping the recording, Safari produces an `audio/mp4` container and Chrome produces `audio/webm;codecs=opus` . To eliminate cross-browser audio decoding failures on the backend, `AudioRecorder.tsx` uses an in-browser `AudioContext.decodeAudioData` pipeline to decode the raw binary stream and re-encode it into a standard 16-bit PCM uncompressed mono `.wav` Blob (`audioBufferToWav`).
 3. `client.ts` (`predict`) packages the blob into a `multipart/form-data` payload containing:
 - `file` : Audio binary payload (named `recording_sample.wav` or original uploaded filename).
 - `test_id` : Optional user/clinician reference string (e.g., `PT-2026-0814`).
 - `source` : `"recording"` | `"upload"` .
- **Output:** HTTP `POST http://localhost:8000/api/v1/predict` multipart request.
- **Why Chosen:** Client-side PCM transcoding guarantees clean, standard RIFF headers, avoiding ffmpeg dependencies for browser audio streams and ensuring seamless cross-browser compatibility across Safari, Chrome, Firefox, and mobile browsers.

Step 2: Backend Entry & Audio Input Validation

- **Executing Files:** `backend/app/api/predict.py` , `backend/app/services/audio_validation.py`
- **Underlying Technology:** FastAPI, Pydantic, Python `soundfile` , `librosa` .
- **Input:** `UploadFile` stream, `source: str` , `test_id: Optional[str]` .
- **Processing Logic:**
 1. `validate_audio_file()` runs multi-stage rejection gates before passing data to PyTorch:
 - **Filename & Extension Gate:** Verifies the file extension is in `settings.allowed_formats_list` (`wav` , `mp3` , `flac` , `m4a` , `ogg` , `webm` , `mp4` , `aac`). Rejects with `400 Bad Request` if invalid.
 - **File Size Gate:** Verifies `0 < len(audio_bytes) <= max_upload_mb * 1024 * 1024` (default limit: 25 MB).
 - **Header & Codec Decoding Gate:** Attempts in-memory decoding via `soundfile.read(io.BytesIO(audio_bytes))` . If in-memory decoding fails (e.g., for container formats), it writes to a `tempfile.NamedTemporaryFile` and decodes via `librosa.load()` using OS-level decoders (e.g., macOS CoreAudio `ExtAudioFile`).
 - **Duration Gate:** Computes `duration_sec = len(waveform) / sample_rate` . Rejects audio shorter than 0.5 s (`min_duration_sec`) with `400 Bad Request` .
 - **Acoustic Energy Gate (Silence Rejection):** Computes `peak = max(|x|)` and `RMS = sqrt(mean(x^2))` . Rejects audio where `peak < 10^{-5}` or `RMS < 10^{-6}` to prevent processing empty or dead microphone inputs.
 - **Output:** Decoded mono floating-point numpy array `waveform` (`float32`), `sample_rate` (`int`), and `duration_sec` (`float`).
 - **Why Chosen:** Defensive validation prevents corrupt audio, silent files, and malicious payload sizes from exhausting GPU/CPU memory or destabilizing neural feature extractors.

Step 3: Acoustic Preprocessing & Standardization

- **Executing Files:** `ml/preprocessing/audio_preprocessor.py` , `backend/app/services/inference.py`
- **Underlying Technology:** `librosa` , `numpy` .
- **Input:** Raw audio waveform numpy array (`float32`), native sample rate.
- **Processing Logic:** Follows `preprocess_config.json` parameters:
 1. **Resampling:** Resamples from native sample rate to 16,000 Hz using high-quality polyphase filtering (`librosa.resample`).
 2. **Top-dB Silence Trimming:** Trims non-speech lead-in and trailing silence below 30 dB relative to peak power (`librosa.effects.trim(top_db=30)`).
 3. **Peak Normalization:** Normalizes amplitude to [-1.0, 1.0] via $x_{\text{norm}} = (x / \max(|x|) + 10^{-8})$.
 4. **Fixed-Length Temporal Segmentation:** Forces audio into exactly 4.0 seconds (64,000 samples at 16 kHz):
 - If duration > 4.0 s, slices the center 4.0 s window.

- If duration < 4.0 s, cyclically tiles/repeats the signal (`pad_mode="repeat"`) until exactly 64,000 samples are reached.
- **Output:** Standardized 1D tensor/array of shape (64000,) (float32).
- **Why Chosen:** Self-supervised speech models expect stationary sampling rates (16 kHz) and fixed temporal receptive fields to generate consistent token sequence lengths.

Step 4: Upstream Self-Supervised Feature Extraction

- **Executing Files:** `ml/preprocessing/feature_extractor.py` , `backend/app/services/inference.py`
- **Underlying Technology:** Hugging Face `transformers` , PyTorch (`torch.no_grad()`).
- **Input:** Standardized audio waveform tensor (1, 64000).
- **Processing Logic:**
 1. Loads `microsoft/wavlm-base-plus` upstream acoustic foundation model (12 Transformer layers, 768 hidden dimension, ~94.7M parameters).
 2. The foundation model is **strictly frozen** (`param.requires_grad = False` , evaluated in `eval()` mode).
 3. Extracts the final hidden states from WavLM across time frames:
 $\$WavLM(x) \text{ in } R^{(B \times T \times F)} = R^{(1 \times 199 \times 768)}$
Where T = 199 temporal frames (each frame spans ~ 20 ms) and F = 768 embedding dimensions.
- **Output:** Dense feature tensor of shape (1, 199, 768).
- **Execution Time:** ~ 80–180 ms on modern CPU.
- **Why Chosen:** WavLM-Base-Plus was pre-trained on 960 hours of LibriSpeech and 60k hours of Libri-Light with masked speech denoising, capturing microscopic fundamental frequency perturbations (jitter), amplitude perturbations (shimmer), and harmonic-to-noise ratios without requiring large-scale labeled clinical training data.

Step 5: Downstream Neural Classifier Inference

- **Executing Files:** `ml/model_def/model.py` , `ml/model_def/convnextv2_stem.py` , `ml/model_def/transformer_encoder.py` , `backend/app/services/inference.py`
- **Underlying Technology:** PyTorch TorchScript (`torch.jit.load`), custom PyTorch architecture.
- **Input:** Feature tensor (1, 199, 768).
- **Architecture Stages & Parameters (6,417,121 trainable parameters):**
 1. **Stage 1 — Atto-Scale ConvNeXt V2 Stem:**
 - Reshapes input to (1, 1, 199, 768).
 - 3-stage depthwise-separable convolutional pyramid with Global Response Normalization (GRN):
 - Stage 1: Channel depth 48, depth 2.
 - Stage 2: Channel depth 96, depth 2, temporal downsampling stride 2.
 - Stage 3: Channel depth 192, depth 4, temporal downsampling stride 2.
 - Output tensor shape: (1, 192, 50, 48) (downsampled temporally by 4x to N = 50 tokens).
 2. **Stage 2 — Spatial Flattening & Linear Projection:**
 - Flattens frequency dimension: 192 x 48 = 9,216 features per token.
 - Linear projection: $R^{9216} \rightarrow R^{256}$ (D = 256).
 - Adds learned 1D positional embeddings: (1, 50, 256).
 3. **Stage 3 — Transformer Encoder Stack:**
 - 3 Transformer Encoder layers, each with 4 attention heads ($d_{\text{head}} = 64$), hidden dimension D = 256, feed-forward network dimension $d_{\text{ffn}} = 1024$, GELU activations, and dropout 0.3.
 - Captures long-range temporal dysphonic patterns across the 50 tokens.
 4. **Stage 4 — Attention Pooling Head:**
 - A single learnable query vector q in $R^{(1 \times 256)}$ computes scaled dot-product attention over the 50 encoder key tokens K in $R^{(50 \times 256)}$:
 $\alpha = \text{softmax}(\frac{q K^T}{\sqrt{256}})$ in $R^{(1 \times 50)}$
 - Computes the pooled representation $v_{\text{pooled}} = \alpha V$ in $R^{(1 \times 256)}$.
 5. **Stage 5 — Classification MLP Head:**

- Linear(256, 128) -> GELU -> Dropout(0.3) -> Linear(128, 1) -> logit in R^1 .

6. Probability & Decision Threshold:

- Sigmoid probability: $p = \text{sigma}(\text{logit}) = (1 / 1 + e^{-\text{logit}})$.
- Binary decision: If $p \geq 0.55$ (calibrated operational threshold), prediction is "parkinsons_risk_indicated"; else "low_risk_indicated".
- Risk tier mapping: Minimal ([0.0, 0.25)), Low ([0.25, 0.55)), Moderate ([0.55, 0.75)), High ([0.75, 1.0]).
- **Output:** Scalar logit, scalar probability p , classification label, and raw 1D attention weights α in R^{50} .
- **Why Chosen:** The ConvNeXt V2 stem compresses the high-dimensional WavLM frequency map into compact temporal tokens, while the Transformer encoder and attention-pooling head learn which phonation segments contain acoustic instability without losing temporal provenance.

Step 6: Explainability Rollout & Time-Alignment

- **Executing Files:** `ml/explainability/attention_rollout.py`, `backend/app/services/inference.py`
- **Underlying Technology:** `numpy`, Linear 1D interpolation.
- **Input:** 1D Attention weights tensor α in R^{50} , target frame count $T = 199$, audio duration 4.0 s.
- **Processing Logic:**
 1. Maps the $N = 50$ downsampled token coordinates $x_{\text{src}} = \text{linspace}(0, 4.0, 50)$ onto the physical $T = 199$ WavLM time coordinates $x_{\text{target}} = \text{linspace}(0, 4.0, 199)$ via 1D linear interpolation:
 $\tilde{\alpha}(t) = \text{interp}(x_{\text{target}}, x_{\text{src}}, \alpha)$
 2. Applies min-max normalization to rescale the salience curve strictly to [0.0, 1.0]:
 $\alpha_{\text{norm}}(t) = \frac{\tilde{\alpha}(t) - \min(\tilde{\alpha})}{\max(\tilde{\alpha}) - \min(\tilde{\alpha})} + 10^{-8}$
 3. Identifies the peak focus timestamp $t_{\text{peak}} = \text{argmax}(\alpha_{\text{norm}})$.
 4. Packages into a JSON-serializable dictionary:

```
json { "timestamps_sec": [0.0, 0.0201, 0.0402, ..., 4.0], "attention": [0.12, 0.14, 0.88, ..., 0.05],
"peak_timestamp_sec": 1.42, "downsample_factor": 4, "num_frames": 199 }
```
- **Output:** Formatted `AttentionHeatmapData` dictionary.
- **Why Chosen:** Provides milliseconds-accurate temporal attribution indicating which portions of sustained phonation triggered classifier activation without requiring computationally expensive backpropagation (such as Integrated Gradients) during live inference.

Step 7: Persistence & Privacy Guarantees

- **Executing Files:** `backend/app/db/repository.py`, `backend/app/models/test_record.py`
- **Underlying Technology:** SQLAlchemy 2.0, SQLite (`backend/app.db`).
- **Input:** Test record metadata, prediction results, JSON-serialized attention heatmap.
- **Processing Logic:**
 1. Generates a unique UUID4 identifier (`rec.id = str(uuid.uuid4())`).
 2. Creates a new `TestRecord` row in SQLite with columns:
 - `id: VARCHAR(36)` (Primary Key)
 - `created_at: DATETIME` (UTC timestamp)
 - `test_id: VARCHAR(100)` (Optional patient reference)
 - `source: "recording" | "upload"`
 - `prediction: "parkinsons_risk_indicated" | "low_risk_indicated"`
 - `probability: FLOAT`
 - `threshold_used: 0.55 (FLOAT)`
 - `audio_duration_sec: FLOAT`
 - `attention_heatmap_json: TEXT` (JSON string containing 199 timestamps and salience values)
 - `report_json: NULL` (populated later on demand)
 - `model_version: "colab-t4-run-20260922"`
 3. **Privacy Guarantee:** Raw audio bytes and decoded PCM arrays are **never written to disk or database**. They exist solely in volatile RAM during the HTTP request lifecycle and are automatically deallocated upon route return.

- **Output:** Persisted `TestRecord` database entity.
-

Step 8: RAG Evidence Retrieval

- **Executing Files:** `backend/app/services/rag.py`, `backend/app/services/report.py`
 - **Underlying Technology:** `chromadb` (Persistent in-process client), `sentence-transformers` (`BAAI/bge-small-en-v1.5`).
 - **Input:** `PredictionResult` object.
 - **Processing Logic:**
 1. `_build_retrieval_query()` formulates a domain-targeted semantic search string based on prediction outcome:
 - If elevated risk: `"vocal and speech changes in Parkinson's disease hypophonia monotone pitch articulation screening indicators"`
 - If low risk: `"vocal speech health normal acoustic baseline Parkinson's screening limitations clinical evaluation"`
 2. Embeds the query using `bge-small-en-v1.5` into a 384-dimensional dense vector.
 3. Queries the persistent ChromaDB collection `pd_knowledge_base` (containing 21 peer-reviewed clinical chunks) using cosine similarity:
`results = collection.query(query_embeddings=[vec], n_results=4)`
 4. Formats results into 4 `EvidenceChunk` objects containing passage text, source institution name, source URL, and similarity score.
 - **Output:** List of 4 structured `EvidenceChunk` objects.
 - **Why Chosen:** In-process ChromaDB with BGE embeddings provides deterministic, fast (< 15 ms) retrieval of peer-reviewed clinical facts without sending health queries to external vector services.
-

Step 9: LLM Clinical Report Synthesis & Defense-in-Depth Guardrails

- **Executing Files:** `backend/app/services/report.py`, `backend/app/services/llm.py`
 - **Underlying Technology:** `openai` (Groq API client), `google-generativeai`, or `ollama`.
 - **Active Configuration:** Groq API with `qwen/qwen3.8-27b` (Base URL: `https://api.groq.com/openai/v1`).
 - **Input:** `PredictionResult`, list of 4 `EvidenceChunk` objects.
 - **Processing Logic:**
 1. Formulates the mandatory system prompt enforcing non-diagnostic vocabulary and evidence grounding.
 2. Constructs the user prompt embedding the exact numerical probability score, classification threshold, risk tier, audio duration, and retrieved evidence passages.
 3. Executes generation at low temperature ($T = 0.2$) with a 1024 token limit.
 4. **Defensive Parsing:** `_parse_llm_json()` strips markdown code fences (`json ...`) and parses the JSON dictionary containing `"screening_summary"` and `"explanation"`. If parsing fails, retries once with an explicit formatting instruction.
 5. **Safety Blocklist Scan:** `_safety_check_text()` scans both output text fields case-insensitively against forbidden diagnostic assertions:
`python FORBIDDEN_DIAGNOSTIC_PHRASES = [ "you have parkinson", "you have pd", "diagnosed with", "confirmed case", "confirmed diagnosis", "positive diagnosis", "we diagnose", "has been diagnosed", "patient has parkinson", "definitely has", "diagnostic proof", "confirms parkinson" ]`
 6. **Safe Fallback Substitution:** If any forbidden phrase is detected, or if the LLM is offline (or returned invalid JSON), the service automatically discards the generated text and injects verified, medically sound fallback prose (`SAFE_FALLBACK_EXPLANATION`).
 7. **Immutable Disclaimer:** Appends the hardcoded, non-modifiable `CLINICAL_DISCLAIMER` string.
 8. Caches the synthesized `Report` JSON into SQLite (`TestRecord.report_json`) so future views load instantaneously without re-invoking the LLM.
 - **Output:** Validated `Report` Pydantic model.
-

Step 10: Response Assembly & Frontend Rendering

- **Executing Files:** `frontend/src/pages/Result.tsx`, `frontend/src/pages/Report.tsx`, `frontend/src/components/AttentionHeatmap.tsx`, `frontend/src/components/ReportSection.tsx`
- **Underlying Technology:** React 18, React Router v6, Tailwind CSS.
- **Rendering Workflow:**
 1. **Result View (`/result/:id`):**
 - Renders status badge pairing color and explicit text (`ELEVATED ACOUSTIC RISK INDICATED` / `LOW ACOUSTIC RISK INDICATED`).
 - Displays calibrated probability percentage (e.g., `82.0%`) alongside decision threshold (`55.0%`).
 - `AttentionHeatmap.tsx` renders the 199-frame temporal intensity bar using `signal-gold` alpha-luminance scaling (`rgba(217, 165, 92, alpha)`), interactive hover inspector, and synchronized audio playback scrubber.
 - Provides CTA button to generate or view the full clinical decision support report.
 2. **Report View (`/report/:id`):**
 - Displays four structured sections with sentence-case headings:
 1. **Model prediction:** Numerical score, decision threshold, and risk tier.
 2. **Retrieved reference information:** 4 cited literature cards with direct external links.
 3. **AI-generated explanation:** Screening summary and acoustic feature context.
 4. **Important disclaimer:** Non-collapsible, high-contrast banner (`border-l-4 border-l-risk-caution`) with text contrast ratio of 18.36:1 against `bg-void` .
 - Includes "Copy report as text" action for structured clipboard export and clean print layout styling.

Step 11: Longitudinal History & Audit Trail

- **Executing Files:** `frontend/src/pages/History.tsx`, `backend/app/api/history.py`, `backend/app/db/repository.py`
- **Underlying Technology:** SQLite indexed queries, React state pagination.
- **Workflow:**
 1. On mount, `History.tsx` calls `listHistory(limit, offset)` targeting `GET /api/v1/history?limit=10&offset=0` .
 2. Renders an editorial table with columns: Timestamp (local timezone), Patient/Session ID, Modality badge (`Live` / `Upload`), Non-Diagnostic Outcome badge, Acoustic Score, and Report Ready indicator.
 3. Clicking any row navigates to `/result/:id` or `/report/:id` , reconstructing full visualizations directly from stored SQLite JSON strings with zero re-inference or LLM token expenditure.

2. Technology Inventory Table

Package / Library	Version	Specific Project Purpose	License
FastAPI	<code>^0.110.0</code>	Asynchronous REST API routing, request validation, exception shielding, and OpenAPI documentation generation.	MIT
Uvicorn	<code>^0.28.0</code>	High-performance ASGI production server hosting FastAPI on <code>http://127.0.0.1:8000</code> .	BSD-3-Clause
Pydantic	<code>^2.6.0</code>	Request and response schema definition, type enforcement, and serialization.	MIT

Package / Library	Version	Specific Project Purpose	License
PyTorch	<code>^2.0.0</code>	TorchScript classifier runtime, tensor math, ConvNeXt V2 convolutional blocks, and attention pooling.	BSD-3-Clause
Transformers	<code>^4.38.0</code>	Loading and running the frozen upstream feature extractor <code>lm-base-plus</code>	Apache 2.0
Librosa	<code>^0.10.0</code>	Audio resampling (16kHz), top-dB silence trimming, and polyphase signal filtering.	ISC
SoundFile	<code>^0.12.0</code>	Fast in-memory decoding and encoding of 16-bit PCM WAV audio streams.	BSD-3-Clause
ChromaDB	<code>^0.4.22</code>	In-process, disk-persisted vector database storing and querying clinical literature embeddings.	Apache 2.0
Sentence-Transformers	<code>^2.5.0</code>	Running the local <code>dense-embedding-model/1.5</code> (384-dim).	Apache 2.0
OpenAI (Python SDK)	<code>^1.14.0</code>	HTTPS client interfacing with Groq's high-speed LPU inference API for report synthesis.	Apache 2.0
Google-GenerativeAI	<code>^0.4.0</code>	Alternative hosted LLM provider interface for Gemini 2.5 Flash.	Apache 2.0
Ollama	<code>^0.1.7</code>	Local offline LLM provider client interfacing with on-device <code>qwen2.5:3b-instruct</code> .	MIT
SQLAlchemy	<code>^2.0.0</code>	Python ORM managing SQLite database schema, connections, and audit log queries.	MIT
Pytest	<code>^8.0.0</code>	Automated unit and integration testing suite for backend API, ML, and RAG services.	MIT
React	<code>^18.3.1</code>	Declarative UI component library powering the clinical frontend application.	MIT
Vite	<code>^6.0.1</code>	Ultra-fast frontend development server and Rollup production build bundler.	MIT

Package / Library	Version	Specific Project Purpose	License
TypeScript	^5.7.2	Static type safety across all React components, API clients, and data interfaces.	Apache 2.0
Tailwind CSS	^3.4.16	Utility-first CSS framework implementing the dark editorial design system tokens.	MIT
Lucide-React	^1.16.0	Consistent, accessible iconography across navigation, actions, and status badges.	ISC
Vitest	^2.1.8	Component and integration unit test runner for the frontend React test suite.	MIT

3. Database Schema

The relational database is stored locally in SQLite at `backend/app.db` under table `test_records`:

```
CREATE TABLE test_records (  
  id VARCHAR(36) NOT NULL PRIMARY KEY,           -- UUID4 unique session identifier  
  created_at DATETIME NOT NULL,                   -- UTC ISO timestamp of screening creation  
  test_id VARCHAR(100),                           -- Optional user or clinician reference tag  
  source VARCHAR(20) NOT NULL,                   -- 'recording' (live mic) or 'upload' (file)  
  prediction VARCHAR(50) NOT NULL,               -- 'parkinsons_risk_indicated' | 'low_risk_indicated'  
  probability FLOAT NOT NULL,                    -- Calibrated neural network risk probability [0.0, 1.0]  
  threshold_used FLOAT NOT NULL,                 -- Operational decision threshold (default: 0.55)  
  audio_duration_sec FLOAT NOT NULL,             -- Length of input audio analyzed in seconds  
  attention_heatmap_json TEXT NOT NULL,          -- JSON string containing 199 temporal timestamps & salience  
  report_json TEXT,                              -- JSON string of synthesized report (NULL until generated)  
  model_version VARCHAR(100) NOT NULL            -- Model checkpoint identifier (e.g. colab-t4-run-20260922)  
);  
  
CREATE INDEX ix_test_records_created_at ON test_records (created_at DESC);  
CREATE INDEX ix_test_records_test_id ON test_records (test_id);
```

4. API Reference

All API routes are prefixed under `/api/v1` on `http://localhost:8000`:

Method	Route	Request Type / Body	Response Status & Shape	Purpose
GET	/api/v1/health	None	200 OK { "status": "ok", "model_artifact_found": true, "timestamp": "..." }	Liveness check and model checkpoint verification.

Method	Route	Request Type / Body	Response Status & Shape	Purpose
POST	/api/v1/predict	multipart/form-data - file: Binary audio - source: "recording" \ \n"upload" - test_id: Optional[str]	200 OK PredictResponse (test_record_id, prediction, probability, threshold_used, attention) 400 Bad Request on audio validation failure.	Ingests audio, executes feature extraction, neural classification, and returns attention heatmap.
POST	/api/v1/report/{id}	Path param id: str	200 OK Report (model_prediction, retrieved_evidence, generated_explanation, clinical_disclaimer) 404 Not Found if record does not exist. 503 Service Unavailable if LLM is offline and fallback disabled.	Generates and caches RAG-grounded AI clinical decision support report.
GET	/api/v1/report/{id}	Path param id: str	200 OK Report 404 Not Found if report not yet generated.	Retrieves cached report without re-invoking LLM.
GET	/api/v1/history	Query params: - limit: int = 50 - offset: int = 0	200 OK List[TestRecordListItem]	Returns paginated list of historical screening sessions.
GET	/api/v1/history/{id}	Path param id: str	200 OK TestRecordResponse	Retrieves full screening record including stored attention heatmap and report JSON.

5. Process Execution Architecture ("What Runs Where")

Process Name	Runtime Command	Host & Port	Role & Dependencies
FastAPI Backend Server	<code>.venv/bin/uvicorn backend.app.main:app --port 8000 --host 127.0.0.1</code>	<code>http://127.0.0.1:8000</code>	Serves REST API endpoints, PyTorch neural inference, ChromaDB RAG vector search, and SQLite ORM queries.
Vite Frontend Dev Server	<code>npm run dev</code> (in <code>frontend/</code>)	<code>http://localhost:5173</code>	Serves the React + TypeScript single-page application with hot module reloading.
Hosted LLM API (Default)	External HTTPS (<code>api.groq.com</code>)	<code>https://api.groq.com/openai/gpt-4o</code> (configured via <code>LLM_API_KEY</code> in <code>.env</code>).	High-speed hosted LPU inference executing <code>gpt-4o</code> (configured via <code>LLM_API_KEY</code> in <code>.env</code>).

Process Name	Runtime Command	Host & Port	Role & Dependencies
Local Ollama Server (Optional)	<code>ollama serve</code> (Air-gapped mode)	<code>http://localhost:11434</code>	Optional local fallback hosting <code>qwen2.5:3b-instruct</code> when internet access is unavailable.

6. Deviations From Original Plan

- Hosted Cloud LLM Integration (Groq & Gemini) alongside Local Ollama:**
 - *Original Plan:* Planned exclusively for local Ollama running `qwen2.5:3b-instruct` on the user's workstation.
 - *Actual Implementation:* Implemented a multi-provider LLM adapter (`LLMService`) supporting Groq (`qwen/qwen3.8-27b`), Google Gemini (`gemini-2.5-flash`), OpenRouter, and Ollama.
 - *Rationale:* Running local LLMs requires 8–16 GB of dedicated RAM/VRAM and several gigabytes of model downloads. Integrating Groq's free-tier LPU API enabled instantaneous (< 1.0 s) report generation on any standard laptop without hardware overhead, while preserving Ollama as an offline fallback.
- In-Browser Web Audio PCM Transcoding & Filesystem Fallback:**
 - *Original Plan:* Expected incoming audio to be standard WAV format.
 - *Actual Implementation:* Added in-browser `AudioContext.decodeAudioData` PCM WAV transcoding in `AudioRecorder.tsx` and a named temporary file fallback in `audio_validation.py`.
 - *Rationale:* Web browsers (especially Safari on macOS/iOS) record audio in MP4/AAC or WebM/Opus containers rather than WAV. Converting directly in browser memory guarantees pristine 16-bit PCM WAV headers across all operating systems.
- Operational Decision Threshold Calibration (0.55 vs 0.50):**
 - *Original Plan:* Standard binary classification threshold at 0.50.
 - *Actual Implementation:* Calibrated operational decision threshold set to 0.55.
 - *Rationale:* Threshold optimization on the validation split using Youden's J statistic (Sensitivity + Specificity - 1) demonstrated that 0.55 achieved superior specificity (90.91% vs 88.64%) and precision (93.65% vs 92.19%) on held-out test data while maintaining identical recall (92.19%).
- Visual Identity & Design System Evolution:**
 - *Original Plan:* Early prompts outlined a standard light SaaS dashboard with royal blue cards.
 - *Actual Implementation:* Evolved into a dark, editorial aesthetic (`bg-void` `#06070C`, `bg-panel` `#12141F`, `signal-gold` `#D9A55C`, Fraunces serif typography, Inter grotesk, hairline borders, and strict clarity-first composition).
 - *Rationale:* Grounded in the visual metaphor of "Voice Becoming Visible Signal", elevating the platform to clinical gravitas while enforcing strict WCAG AAA contrast (18.36:1\$) on data and safety disclaimers.